

ZAATOT AI PLANE

Product Fit Agent Implementation Guide

From boycott verification to a safe, explainable, personalized product decision

AGENTS | TOOLS | SYNCHRONIZATION | DATA | DELIVERY

A beginner-friendly technical blueprint grounded in the 56-page ZTroop algorithms book and mapped to the current Bun, Elysia, Turso, Redis, VoltAgent, AI SDK, and Zod backend.

1 Verify

Identity and boycott evidence

2 Personalize

Consent, safety, and goals

3 Explain

Confidence and alternatives

Contents

This document translates the supplied ZTroop algorithms book into an implementation path for the current Zaatot backend. It is written for a reader who is new to modern AI systems, but it keeps enough technical precision for engineering planning and code review.

Important status

The source book explicitly describes itself as an implementation-ready blueprint, not a deployed or production-verified system. This guide therefore separates what already exists in the backend from what must be designed, implemented, tested, and approved.

1. Executive idea	4
1.1 What already exists	4
1.2 What must be added	4
2. From scan to personalized decision	5
2.1 End-to-end flow	5
2.2 The decision tree	5
2.3 Do not collapse the result into one score	5
3. Agent operating model	6
3.1 Roles for the first implementation	6
3.2 Which roles may call an LLM?	7
4. Tool calling: how every agent decides	8
4.1 Seven checks before every call	8
4.2 Tool ownership matrix	8
4.3 Tool implementation rules in this repository	8
5. Synchronizing agents safely	9
5.1 Shared execution context	9
5.2 Typed specialist result	10
5.3 Required versus optional roles	10
5.4 Join algorithm	10
6. Requirements for deciding product fit	11
6.1 User inputs: private Store A	11
6.2 Product inputs: shared Store B	11
6.3 Hard constraints before scoring	11
6.4 Soft scoring after hard gates	12
6.5 Confidence is different from fit	12
7. Final decision contract	12
7.1 Outcome rules	12
7.2 Worked example	12
7.3 Explanation prompt boundary	13
8. Data architecture and privacy	13

8.1 Proposed records	13
8.2 Why the current boolean is not enough	13
8.3 Privacy and security boundaries	14
9. Mapping the design into this backend	14
9.1 Proposed folder structure	14
9.2 Service boundaries	14
9.3 Recommended HTTP surface	15
10. Implementation sequence inside one request	15
10.1 Agent file pattern	15
10.2 Boycott resolution change	15
10.3 Configuration to add	15
11. Caching, retries, and background work	16
11.1 Safe ready-answer cache	16
11.2 What remains synchronous	16
11.3 Later Redis Streams contract	16
12. Security, safety, and failure behavior	17
12.1 Prompt injection model	17
12.2 Medical boundary	17
13. Testing and observability	17
13.1 Test pyramid	17
13.2 Minimum golden scenarios	17
13.3 Metrics without personal leakage	18
14. Delivery roadmap	18
14.1 Recommended first backlog	18
15. Definition of done	19
15.1 A new agent is done when	19
15.2 The product-fit feature is done when	19
16. Beginner glossary and reading path	20
16.1 Suggested learning order	20
17. Sources and repository anchors	20

1. Executive idea

Zaatot should answer two different questions in a controlled order. First: **what is this product, and is there credible boycott evidence?** Second: **given consented user requirements, is this product safe and useful for this particular person?** The first question is shared knowledge. The second is private personalization.

The safest architecture is not one powerful model. It is a bounded workflow in which deterministic services retrieve facts and apply hard rules, specialized agents return typed claims, an orchestrator joins the results, and a final gate validates the response. The LLM is most valuable for planning difficult cases and explaining validated results - not for inventing facts or overriding safety.

Use AI where language and bounded reasoning help. Use rules where correctness, privacy, medical caution, authorization, or reproducibility matter.

1.1 What already exists

Existing capability	Current implementation	How it helps
Product identity	scan service + Open Food Facts provider + main product cache	Provides barcode resolution, product metadata, brand, images, NOVA, Eco-Score, and Nutri-Score.
Boycott provider	Boycat encrypted API adapter	Returns campaigns, source links, alternatives, and provider-normalized confidence.
Internal boycott logic	Curated seed + entity matching + confidence formula	Provides a second evidence path and can preserve local policy knowledge.
User isolation	One user Turso database plus tenant access checks	Natural Store A location for profile, consent, private decisions, and history.
History	scan_history and user_list	Can support optional trend and preference signals after consent.
AI foundation	AI SDK message utility, VoltAgent, Google provider, Zod	Dependencies and folder conventions exist, but no production agent is wired.
Coordination	Redis clients and tenant locks	Can support AI budgets, scoped locks, ready-answer cache, and later worker streams.

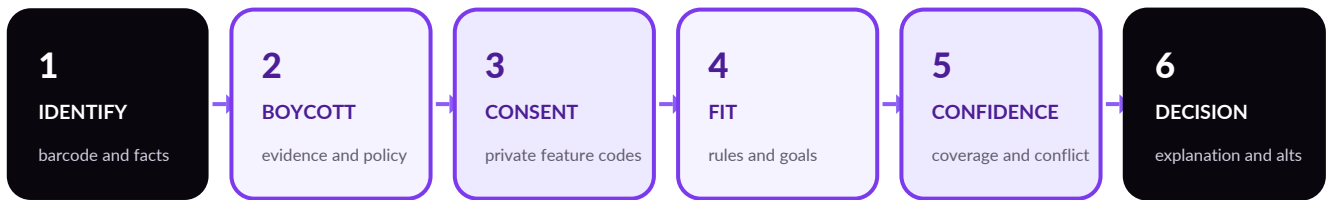
1.2 What must be added

- A consented user profile expressed as minimized feature codes rather than raw personal text.
- A deterministic product-safety and goal-fit engine with versioned rules and configurable thresholds.
- An evidence-rich decision contract that represents boycott, user fit, confidence, warnings, alternatives, and disagreements separately.
- Strict agent tool allowlists and an AI-specific execution context carrying user, tenant, purpose, consent, deadline, budget, and configuration versions.
- Durable decision, evidence, consent, audit, and configuration records plus integration and red-line tests.

Recommended scope Start with one synchronous Consumer pipeline and one optional Storyteller model call. Do not start by deploying the complete 34-role organization from the source book.

2. From scan to personalized decision

2.1 End-to-end flow



The current `service_scan.identify()` already performs the first two steps for barcode scans: product resolution, scan-history creation, and Boycat lookup. The new AI plane should call these services directly rather than calling the HTTP routes from inside the backend.

2.2 The decision tree

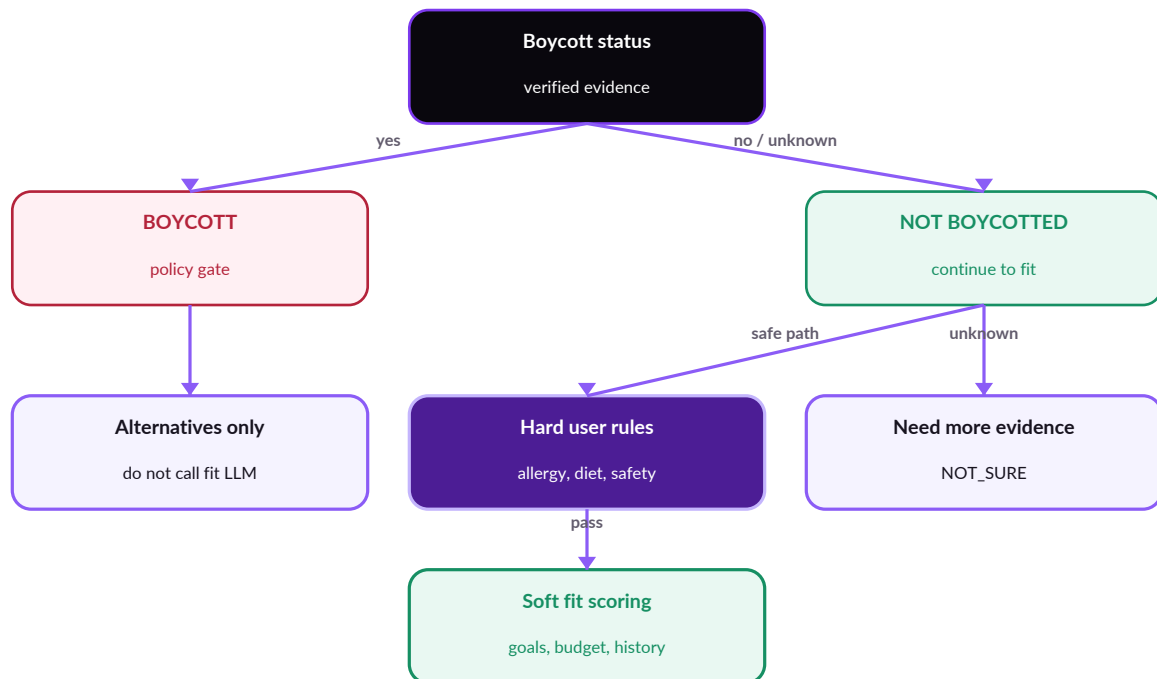


Figure 1. Boycott is resolved before private fit evaluation. A strict business policy can stop a boycotted product and evaluate only safer alternatives. An advisory policy may still show personal dimensions, but the boycott outcome must remain a separate, visible dimension.

2.3 Do not collapse the result into one score

Dimension	Question	Authority	Possible result
Identity	Did we identify the exact product?	Detective + catalog rules	matched / ambiguous / unknown
Boycott	Is there credible active evidence?	Boycott resolution service + policy	boycott / not_boycotted / needs_review / unknown
Safety	Does a hard user restriction conflict?	Deterministic Medic rules	pass / warning / fail / unknown
Goal fit	Does it support declared goals?	Versioned Coach scoring	strong / mixed / weak / omitted
History	How does it compare with the user's trend?	Historian aggregate rules	improves / neutral / worsens / insufficient

Dimension	Question	Authority	Possible result
Final	What should the app show?	Referee + Skeptic + Gatekeeper	recommended / qualified / not_recommended / not_sure / blocked
Policy choice to approve		Decide whether a confirmed boycott is a hard veto, a strong negative dimension, or a user-configurable ethics preference. The LLM must never invent this policy.	

3. Agent operating model

In the source book, an 'agent' may be an LLM, a deterministic function, a SQL/retrieval operation, or a workflow step. That definition is essential: most Zaatot roles should not be model calls. They should be named, testable responsibilities with narrow tools and typed outputs.

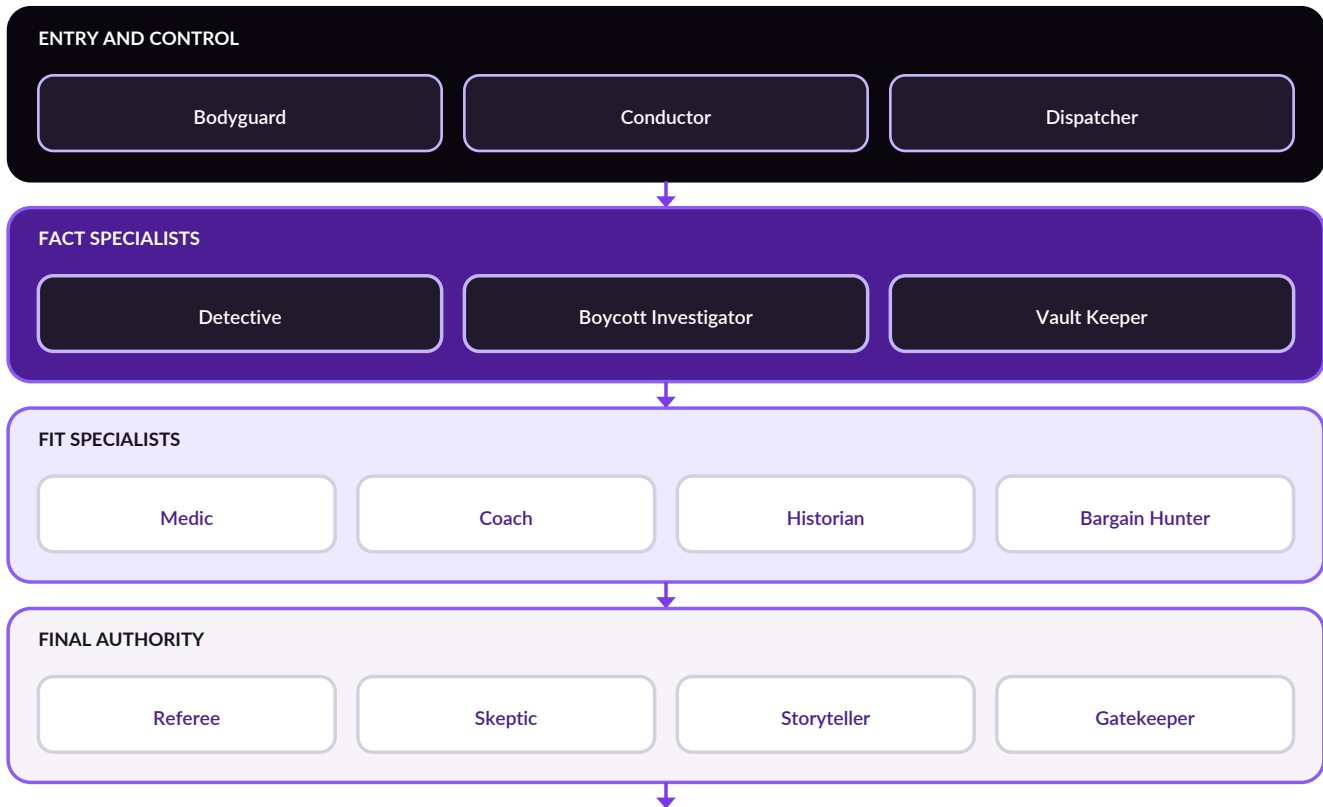


Figure 2. A four-layer request path. Only Conductor and Storyteller may need a model. Safety and final authority remain deterministic.

3.1 Roles for the first implementation

Role	Implementation	Responsibility	Fallback
Bodyguard	Rules/classifier	Validate size/schema; mark user, OCR, RAG, and provider text untrusted	Block or sanitize
Conductor	Workflow supervisor	Create bounded plan; choose required and optional roles	Deterministic routing table
Dispatcher	Deterministic router	Bind role to allowed tool set, budget, deadline, and schema	Rules/retrieval-only
Detective	Retrieval	Identify exact product and return canonical facts with version	Manual selection / NOT_SURE
Boycott Investigator	Retrieval + rules	Merge Boycat and internal knowledge without erasing disagreement	Unknown / needs_review
Vault Keeper	Private retrieval	Return consented feature codes from the user tenant	Generic non-personal mode
Medic	Hard rules	Allergy, dietary, ingredient, and health-adjacent flags	NOT_SURE / hard warning

Role	Implementation	Responsibility	Fallback
Coach	Rules first	Score product facts against declared goals	Omit when no consent or facts
Historian	Aggregate rules	Compare current choice with consented history aggregates	Omit when coverage is thin
Bargain Hunter	Retrieval + ranker	Find non-boycotted, safe, relevant alternatives	No verified alternative
Referee	Hard policy gate	Record material conflict and apply veto rules	BLOCK / NOT_SURE
Skeptic	Confidence calculator	Evaluate source tier, freshness, coverage, directness, and conflict	Low confidence / NOT_SURE
Storyteller	LLM or template	Explain only validated structured results	Localized deterministic template
Gatekeeper	Final rules	Validate schema, consent, evidence, privacy, scope, and wording	BLOCK

3.2 Which roles may call an LLM?

Role	Default	LLM allowed when	LLM must not
Conductor	Deterministic plan	Input is ambiguous and a bounded planner improves routing	Decide product truth or authorize tools
Coach	Versioned rules	Only to summarize mixed goal factors after scoring	Create goals or override hard restrictions
Historian	Aggregate rules	Only to phrase a neutral nudge	Read raw history or judge the user
Storyteller	Template	Structured evidence needs a natural-language explanation	Add facts, citations, or decisions
All gates	Rules only	Never in the MVP	Fail open or accept malformed output

4. Tool calling: how every agent decides

A tool is not simply a function the model can see. It is a protected adapter around an existing service. The agent requests a tool; runtime authorization decides whether the call is allowed. Unknown tools are denied. Agents never receive raw SQL, shell, filesystem, unrestricted HTTP, configuration activation, or cross-tenant administration.

4.1 Seven checks before every call

- **Need:** Is the tool necessary for the current typed task, or can the answer use existing results?
- **Allowlist:** Is the tool ID present in the role's immutable tool package?
- **Identity and scope:** Does the execution context bind the authenticated user, user tenant, and purpose?
- **Consent:** Does a private read have the required current consent snapshot?
- **Minimization:** Are parameters limited to the smallest allowed fields or feature codes?
- **Budget and deadline:** Are tool calls, money, tokens, and remaining wall time available?
- **Fallback:** If the dependency fails, is there an approved deterministic, retrieval, template, NOT_SURE, or BLOCK path?

```
AUTHORIZE_TOOL_CALL(role, tool_id, ctx, input):
  require tool_id in ALLOWLIST[role]
  require ctx.user_id and ctx.tenant_id match authentication
  require ctx.purpose in role.allowed_purposes
  if tool reads private data: require current consent_snapshot_id
  validate input schema and reject unknown fields
  reserve budget and confirm deadline
  execute service adapter; validate output schema
  append audit; release unused budget
  otherwise DENY and audit
```

4.2 Tool ownership matrix

Role	Allowed tool(s)	Data authority	Call trigger	Fallback
Bodyguard	security.validate_input	Request only; no writes	Every AI request	BLOCK
Conductor	config.read_active; budget.reserve	R: config pointers; W: plan audit	Before planning	Routing table
Detective	catalog.lookup_product	R: main product/brand; no writes	Every product decision	Manual choice
Boycott Investigator	boycott.resolve	R: Boycat + internal evidence; no request-path writes	After identity	needs_review
Vault Keeper	profile.read_feature_codes; consent.read	R: user tenant; W: access audit	Personal mode only	Generic mode
Medic	rules.screen_product	R: product facts + feature codes; no writes	Safety tags or restrictions	NOT_SURE
Coach	score.goal_fit	R: product facts + goal codes; no writes	Goals consented	Omit dimension
Historian	history.read_aggregates	R: user tenant aggregates; no writes	History consented	Omit dimension
Bargain Hunter	catalog.search_alternatives	R: shared catalog; no writes	Rejected/qualified result	No verified alt
Referee	policy.evaluate; disagreement.record	R: typed claims; W: disagreement transaction	Before explanation	BLOCK
Skeptic	confidence.calibrate	R: evidence metadata; no writes	Every live/cache result	Low / NOT_SURE
Storyteller	explain.render	R: validated claims only; no writes	Explanation enabled	Template
Gatekeeper	decision.validate; decision.commit; cache.put	R: typed results; W: decision/evidence/audit + cache	Final step	BLOCK

4.3 Tool implementation rules in this repository

- Create tools under `src/ai/tool/<module>/` with a Zod DTO file and a VoltAgent tool file, matching the project skill.
- Each tool calls a service such as `service_scan`, `service_boycott_resolution`, `service_profile`, or `service_product_fit`. Tools do not query Drizzle directly.

- Extend or replace the current `lib_tool_check_context` for AI. Its present contact-oriented shape expects `contact_phone` and returns `user_id: -1`, which is not safe for personalized AI decisions.
- Every tool returns a typed success/failure object with stable machine codes. Free-form exceptions are normalized by the service or controller boundary.
- Provider adapters are tools only through a service that enforces timeouts, allowed destinations, response validation, and evidence metadata.

```
export const tool_catalog_lookup = createTool({
  name: 'tool_catalog_lookup',
  parameters: dto_tool_catalog.lookup,
  execute: async ({ barcode }, options) => {
    const ctx = lib_tool_check_ai_context(options, 'scan_decision')
    if (!ctx.success) return ctx
    return service_product_fit.lookup_product({ barcode }, ctx.payload)
  },
})
```

5. Synchronizing agents safely

Synchronization is a workflow problem, not a group chat between models. Agents exchange typed result objects through the Conductor. Specialists never call each other arbitrarily, never write the final decision independently, and never rely on shared prompt memory as durable state.

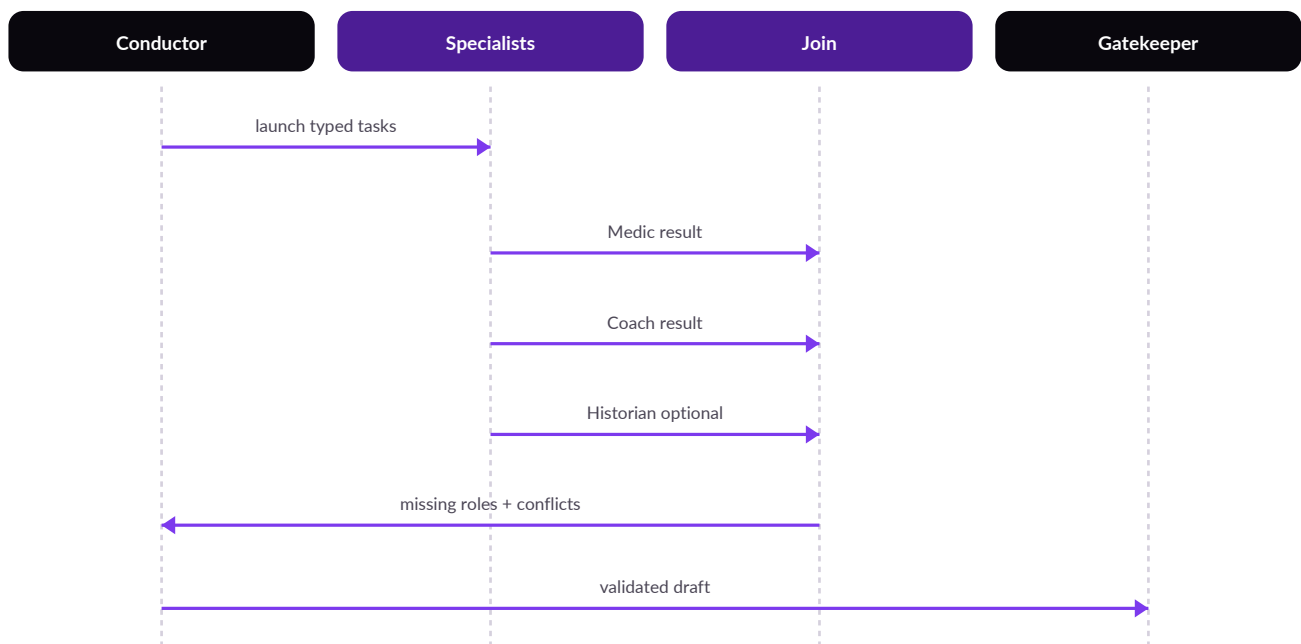


Figure 3. Bounded fan-out and join. Required roles must finish or force a safe fallback; optional roles may be cancelled at the deadline.

5.1 Shared execution context

```
ExecutionContext {
  request_id, trace_id, tenant_id, user_id, purpose,
  consent_snapshot_id, locale, config_versions,
  deadline_at, budget: { tokens, money, tool_calls, wall_time_ms },
  allowed_tools
}
```

The context is created by the authenticated API, not by a model. A tool must not accept a tenant or user ID supplied only in model text. The runtime injects identity and purpose into VoltAgent context and verifies them again inside each sensitive tool.

5.2 Typed specialist result

```
AgentResult<T> {
  status: success | partial | not_sure | blocked | retryable_failure | permanent_failure
  value: T | null
  claims: Claim[]
  evidence: EvidenceRef[]
  warnings: Warning[]
  fallback_used: string | null
  config_versions, started_at, finished_at
}
```

5.3 Required versus optional roles

Condition	Required	Optional	If missing
Every decision	Bodyguard, Detective, Boycott, Referee, Skeptic, Gatekeeper	Storyteller	NOT_SURE or BLOCK depending on role
Personal mode	Vault Keeper	Historian	Generic mode only if disclosed and safe
Allergy/diet/health code	Medic	None	Do not recommend; return NOT_SURE or warning
Goals available	Coach	Historian	Omit goal dimension; never infer goals
Rejected or qualified	None	Bargain Hunter	Return result without alternative

5.4 Join algorithm

- Launch independent specialists with separate budgets after identity and boycott policy permit personalization.
- Wait only until the join deadline. Cancel late optional work and record it as missing coverage.
- A missing safety-critical result produces NOT_SURE or BLOCK. A missing optional result removes that dimension; it never becomes a neutral score.
- Normalize claims by subject and predicate, then send supporting and opposing positions to the Referee.
- Calculate confidence from source tier, freshness, directness, coverage, and unresolved conflict - not from how many agents agree.
- Only Gatekeeper commits decision, evidence, disagreement, consent snapshot, configuration versions, and audit in one transaction where possible.

Synchronization anti-pattern

Do not let agents write to the same mutable JSON object, call each other recursively, or communicate only through prose. Those patterns make authorization, timeouts, replay, and testing unreliable.

6. Requirements for deciding product fit

A model cannot determine user suitability from a barcode alone. The backend needs a complete, consented, and versioned set of product facts, user feature codes, policy rules, and evidence metadata.

6.1 User inputs: private Store A

Category	Examples	Use	Privacy rule
Consent	scan decision, health screen, goals, history	Controls every personal read	Immutable snapshot; recheck before commit
Hard restrictions	allergy:peanut, diet:gluten_free, avoid:alcohol	Deterministic pass/fail	Feature codes only; no raw notes
Health-adjacent codes	pregnancy flag, medication interaction class	Cautious rule screening	Explicit consent; never diagnosis
Goals	low_sugar, high_protein, lower_sodium, minimally_processed	Soft goal-fit scoring	Declared by user; never inferred
Preferences	halal, vegan, organic, local, palm-oil-free	Hard or soft by user setting	Purpose-limited
Ethics policy	strict boycott veto or advisory	Controls boycott gate	Explicit, configurable policy
Budget and locale	price band, country, availability radius	Alternative ranking	Coarse values when possible
History aggregate	category frequency, trend windows	Neutral longitudinal nudge	Aggregate only; separate consent

6.2 Product inputs: shared Store B

- Exact identity: barcode, product ID, name, brand, region, package variant, and source version.
- Ingredients and normalized allergens, including whether the list is complete and when it was observed.
- Nutrition per 100g and per serving: energy, sugar, protein, fat, saturated fat, sodium, fiber, and serving size.
- Classification signals such as NOVA, Nutri-Score, Eco-Score, product type, and dietary labels.
- Boycott evidence: claim status, entity match path, source identity, source tier, observation date, freshness, and disputes.
- Alternative candidates with category, safety facts, location, availability, price freshness, and boycott status.

Current data gap

The existing product table stores ingredients, allergens, NOVA, Eco-Score, and Nutri-Score, but a reliable fit engine should add structured nutrient values, serving information, evidence dates, and completeness flags.

6.3 Hard constraints before scoring

Hard gate	Example	Result
Identity	Two products match the same text input	Ask the user to choose; do not guess
Boycott policy	Confirmed boycott + strict user policy	not_recommended; evaluate alternatives
Allergy	Peanut allergen + peanut allergy code	not_recommended with hard warning
Diet	Alcohol ingredient + avoid:alcohol	not_recommended
Missing safety facts	No ingredient list for an allergy-sensitive user	not_sure
Consent	Health screen requested without consent	generic mode or blocked personal dimension

6.4 Soft scoring after hard gates

Soft fit is a transparent, versioned policy calculation. The following weights are an example configuration, not universal truth: goal fit 35%, nutrition fit 25%, preferences 15%, price/availability 15%, and history trend 10%. Each dimension must expose its inputs and reason.

```
soft_fit = weighted_mean(available_dimensions)
coverage = available_weight / configured_total_weight
if coverage < minimum_coverage: outcome = NOT_SURE
if soft_fit >= recommended_threshold: RECOMMENDED
else if soft_fit >= qualified_threshold: QUALIFIED
else: NOT_RECOMMENDED
```

Missing dimensions are omitted and lower coverage. They are never converted to zero. Exact weights, minimum coverage, and thresholds belong in an immutable configuration version with tests and an owner.

6.5 Confidence is different from fit

Term	Meaning	Example
Fit	How well the product supports this user's requirements	Weak because sugar conflicts with low_sugar goal
Confidence	How reliable and complete the evidence is	High because nutrition and ingredient facts are fresh and sourced
Coverage	How much of the required evaluation could run	70% because live price and history were unavailable
Disagreement	Material specialists or sources conflict	Ownership source is disputed; boycott result needs review

7. Final decision contract

The API should return structured evidence first and a human explanation second. Clients must not parse free-form model prose to discover whether a product is safe or recommended.

```
ProductDecision {
  decision_id, request_id, product_id, user_id,
  outcome: recommended | qualified | not_recommended | not_sure | blocked,
  boycott: { status, policy_effect, confidence, evidence_ids },
  dimensions: [{ name, result, score?, reason, evidence_ids }],
  reasons, warnings, alternatives,
  disagreements: [{ claim, positions, severity, resolution }],
  confidence: { band, reason, coverage, freshness, unresolved_conflicts },
  evidence, consent_snapshot_id, config_versions, cache, created_at
}
```

7.1 Outcome rules

Outcome	Use when	Client behavior
recommended	Hard gates pass, soft fit passes, evidence coverage is sufficient	Show positive reasons and optional alternatives
qualified	Safe result with material limitations or mixed soft dimensions	Show limitations prominently
not_recommended	Confirmed policy veto, hard user conflict, or poor soft fit	Show reason and verified alternatives
not_sure	Identity, evidence, freshness, coverage, or required specialist is insufficient	Ask for rescan, missing info, or later retry
blocked	Safety, privacy, consent, tenant, schema, or audit integrity fails	Safe machine-coded error; no model prose

7.2 Worked example

Example decision The product is not classified as boycotted with moderate confidence. It is not recommended for this user because the ingredient list declares peanuts and the consented profile contains allergy:peanut. The low-sugar goal also conflicts with 24g sugar per serving. Two non-boycotted, peanut-free alternatives are returned. The allergy rule - not the LLM - determines the outcome.

7.3 Explanation prompt boundary

- Input: validated decision dimensions, supported claims, evidence labels, disagreements, confidence reason, locale, and tone preference.
- No input: raw name, phone, full profile, full scan history, credentials, internal prompts, or unrestricted provider text.
- Output: one Zod-validated explanation object. Unknown fields are rejected.
- If the model fails or invents a claim, use deterministic localized templates.

8. Data architecture and privacy

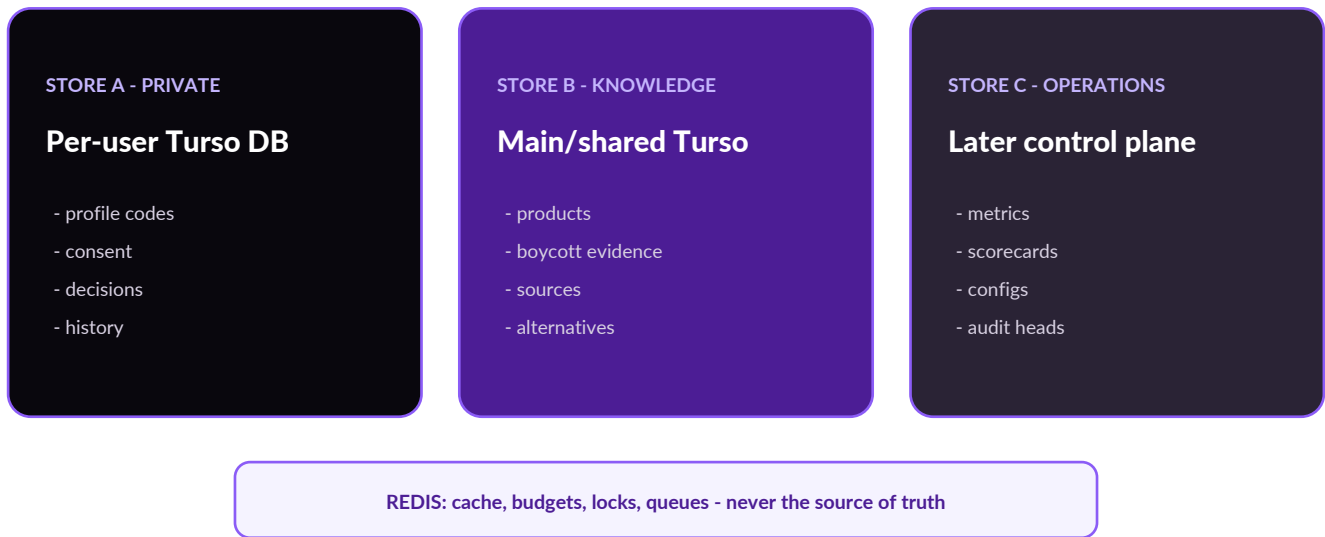


Figure 4. Logical Store A/B/C separation mapped to the current database-per-tenant architecture. Physical Store C separation can wait, but credentials and table authority should be designed now.

8.1 Proposed records

Location	Record/table	Purpose	Write authority
User tenant	ai_user_profile	Encrypted/minimized feature codes and profile version	Profile service only
User tenant	ai_consent	Purpose grants, withdrawal, effective dates, hash	Consent service only
User tenant	ai_decision	Final outcome, dimensions, confidence, versions	Gatekeeper commit path
User tenant	ai_decision_evidence	Evidence references used for one decision	Gatekeeper commit path
User tenant	ai_decision_disagreement	Preserved conflicting claims and resolution	Gatekeeper commit path
User tenant	ai_audit_event	Append-only private request and gate events	Audit writer
Main/shared	ai_boycott_claim	Versioned claims with active/disputed/superseded states	Validation service only
Main/shared	ai_source_document	Source identity, tier, dates, license, freshness	Validated ingestion
Main/shared	ai_product_fact	Normalized nutrient/ingredient facts and provenance	Catalog validation
Control plane	ai_config_version	Immutable rules, weights, routing, freshness, prompts	Approved executor later

8.2 Why the current boolean is not enough

The existing `brand_is_boycotted` snapshot is useful for fast reads, but it cannot represent unknown, `needs_review`, `disputed`, `stale`, `source versions`, or competing evidence. Keep it only as a derived cache. The decision source of truth should be versioned evidence records.

8.3 Privacy and security boundaries

- Private profile and health-adjacent codes stay in the user's tenant database and are read only for an allowed purpose.
- Night/background knowledge workers never receive Store A credentials or user IDs.
- Internal operations receive minimized metrics such as latency, outcome, cache hit, timeouts, and cost - never raw feature codes.
- Personal cache keys use a non-reversible tenant-scoped profile/consent hash and are invalidated on consent withdrawal or profile change.
- Model providers for personal or health-adjacent explanation require controlled retention/no-training terms and redacted observability.
- Prompt injection protection applies to user text, OCR, product descriptions, web pages, and provider content. Untrusted text cannot authorize tools.

Migration blocker

Before adding high-value personal AI tables, replace or harden the current destructive tenant schema synchronization. Versioned, reversible migrations and historical-schema tests are a prerequisite.

9. Mapping the design into this backend

9.1 Proposed folder structure

```
src/
ai/
agent/product-fit/
product-fit.agent.ts
product-fit.prompt.agent.ts
product-fit.schema.agent.ts
agent/storyteller/...
tool/catalog/{catalog.tool.ts,catalog.dto.tool.ts}
tool/boycott/{boycott.tool.ts,boycott.dto.tool.ts}
tool/profile/{profile.tool.ts,profile.dto.tool.ts}
tool/product-fit/{product-fit.tool.ts,product-fit.dto.tool.ts}
tool/utils.tool.ts
module/main/boycott-resolution/...
module/main/product-fit/...
module/user/profile/...
module/user/consent/...
module/user/ai-decision/...
```

This follows the repository's required Agent and Tool file patterns. Business truth remains in services. Agents orchestrate tools; tools authorize and adapt services; services own database and provider behavior.

9.2 Service boundaries

Service	Owns	Must not own
service_product_identity	Normalize input, call catalog/provider, resolve ambiguity	User profile or final recommendation
service_boycott_resolution	Merge Boycat/internal evidence, preserve sources and disagreement	Personal health or goals
service_consent	Grant, withdraw, snapshot, and revalidate purposes	Product scoring
service_profile	Encrypted profile and feature-code projection	Model prompting
service_product_safety	Deterministic hard-rule screen	Diagnosis or free-form reasoning
service_product_fit	Versioned dimension scoring and outcome draft	Final commit without gates
service_alternative	Filter and rank verified alternatives	Recommend products that fail hard gates
service_ai_decision	Commit decision/evidence/audit idempotently	Invent claims

9.3 Recommended HTTP surface

Method	Route	Purpose
PUT	/user/ai-profile	Create/update consented goals, restrictions, and preferences
POST	/user/ai-consent	Grant or withdraw versioned AI purposes
POST	/ai/product-decision	Run the full identity -> boycott -> fit pipeline using Idempotency-Key
GET	/ai/product-decision/:id	Read one authenticated user's persisted decision
POST	/ai/product-decision/:id/feedback	Record useful/not-useful/correction feedback without changing history

Keep /scan/identify as the lower-level product and boycott endpoint. The new AI route composes its services and adds personal evaluation. This avoids breaking current clients and preserves a clear boundary.

10. Implementation sequence inside one request

```

PRODUCT_DECISION(request, payload):
  ctx = resolve_authenticated_ai_context(payload, request.purpose)
  bodyguard.validate(request)
  config = read_active_config(ctx.tenant_id)
  product = product_identity.resolve(request.scan_value)
  boycott = boycott_resolution.resolve(product)
  if boycott triggers strict veto:
    alternatives = alternative.find_safe(product, generic_constraints)
    return gate_and_commit(non_personal_boycott_decision)
  consent = consent.snapshot(ctx.user_id, requested_purposes)
  codes = profile.read_feature_codes(consent)
  run Medic + Coach + optional Historian under independent budgets
  join typed claims; Referee records conflict
  Skeptic calibrates confidence and coverage
  if rejected/qualified: Bargain Hunter finds verified alternatives
  Storyteller renders explanation or template fallback
  Gatekeeper rechecks consent, schema, evidence, privacy, and scope
  commit decision + evidence + disagreement + audit; return JSON

```

10.1 Agent file pattern

The project skill requires each agent to be a VoltAgent Agent with a separate prompt and Zod schema. Keep the wrapper return typed and never return any in the final implementation.

```

export const $agent_storyteller = new Agent({
  name: 'Product Decision Storyteller',
  instructions: prompt_agent_storyteller,
  model: google(process.env.DEFAULT_AI_MODEL_NAME),
  tools: [], // explanation receives structured facts; no DB tools
})

const result = await $agent_storyteller.generateText(message, {
  output: Output.object({ schema: schema_agent_storyteller }),
})

```

10.2 Boycott resolution change

- Unify the current Boycat adapter and internal decision service behind one service_boycott_resolution contract.
- Store each source position independently. Do not average competing claims into a single confident boolean.
- Replace request-time global brand mutation with a validated evidence write or background reconciliation. The current scan flow updates a shared brand snapshot and silently ignores update failures.
- Return freshness, observation time, provider status, match path, and disagreement so Skeptic can calibrate honestly.

10.3 Configuration to add

Group	Examples	Rule
Model	DEFAULT_AI_MODEL_NAME, AI_STORYTELLER_ENABLED	Model key is required only when the model feature is enabled

Group	Examples	Rule
Deadline	AI_REQUEST_TIMEOUT_MS, AI_JOIN_TIMEOUT_MS	Must be lower than the Elysia request timeout
Budget	AI_MAX_TOOL_CALLS, AI_MAX_COST_UNITS	Optional work is shed before safety gates
Policy	AI_BOYCOTT_POLICY, AI_MIN_IDENTITY_CONFIDENCE	Version and audit; do not bury in prompts
Fit	AI_FIT_CONFIG_VERSION	References immutable weights and thresholds
Freshness	AI_BOYCOTT_FRESH_DAYS, AI_PRODUCT_FACT_FRESH_DAYS	Unknown/stale lowers confidence or blocks
Cache	AI_READY_TTL_SECONDS	Artifact also carries evidence and config versions

11. Caching, retries, and background work

11.1 Safe ready-answer cache

A cache hit is an optimization, not permission to skip gates. Use a key such as `ai:decision:{tenant}:{profileHash}:{product}:{configVersion}`. The profile hash must be non-reversible and tenant scoped.

- Every artifact carries tenant/user scope, consent hash, evidence IDs and versions, config versions, freshness, expiry, and artifact hash.
- On read, recheck consent, scope, evidence freshness, current configuration compatibility, Skeptic, and Gatekeeper.
- Invalidate on consent withdrawal, profile change, product fact change, boycott evidence dispute/supersession, or configuration activation.
- Never share a personalized artifact across users, even if their profile codes appear similar.

11.2 What remains synchronous

Synchronous request path	Background path
Authentication, tenant resolution, consent snapshot	Knowledge source ingestion and validation
Product identity and current boycott lookup	Ownership/news/health evidence refresh
Hard user rules and soft fit scoring	Embedding refresh and pre-cooked non-private fragments
Referee, confidence, final gate, commit	Evaluation, scorecards, canaries, and reports

The phone request must never wait for night workers. When knowledge is stale or missing, return a qualified or not-sure result and enqueue a minimized refresh job.

11.3 Later Redis Streams contract

- Turso job/outbox records are the durable source of enqueue intent; Redis Streams are an at-least-once projection.
- Workers validate envelopes, tenant/global scope, purpose, schema version, payload hash, and idempotency key before side effects.
- Acknowledge only after durable effect and audit completion. Reclaim abandoned pending entries and dead-letter permanent failures.
- Never place user names, contact data, raw health/profile values, secrets, arbitrary tools, prompts, SQL, or executable code in jobs.
- Create an urgent stream for recalls and safety supersessions; a priority field cannot jump existing FIFO work.

12. Security, safety, and failure behavior

Failure	Required behavior	Forbidden behavior
Identity ambiguous	Ask for manual selection or rescan	Guess the product
Boycott evidence conflicts	Preserve disagreement; needs_review or qualified	Average sources silently
Ingredients missing for allergy user	NOT_SURE with safe next step	Assume no allergen
Consent withdrawn mid-request	Remove personal dimensions, invalidate cache, block commit	Use the earlier snapshot without recheck
Model outage	Rules and localized template	Skip hard gates
Provider outage	Compatible fresh evidence or unknown	Convert outage into not_boycotted
Audit failure	Block material commit or mark unhealthy	Return unaudited personal decision as complete
Cross-tenant mismatch	Block, alert, preserve redacted evidence	Retry with broader credentials
Budget exhausted	Cancel optional enrichment	Drop Medic, Referee, Skeptic, or Gatekeeper
Malformed model output	Reject; repair deterministically once or template fallback	Return raw completion

12.1 Prompt injection model

- Treat product descriptions, OCR, uploaded text, provider prose, and retrieved documents as data, never instructions.
- Do not place tool names, secrets, or internal policy text in untrusted content fields.
- The Bodyguard validates and labels content before the Conductor. Risky text is either excluded or processed in a sandboxed extraction role with no network, Store A, or write tools.
- Authorization depends on server-side context and allowlists, never on instructions produced by the model.

12.2 Medical boundary

Health-adjacent, not medical authority

The Medic flags ingredient/allergy/medication risks using approved deterministic rules and licensed references where required. It does not diagnose, treat, or weaken a rule because another agent disagrees.

13. Testing and observability

13.1 Test pyramid

Layer	Examples	Proof
Pure unit	feature-code validation, ingredient normalization, hard gates, soft score, confidence	Exact deterministic outcomes
Service integration	Turso profile/consent/decision transaction, Redis cache, Boycat/Open Food Facts fixtures	Real queries and failure classification
Agent contract	Every input/output schema, unknown fields, allowlists, timeout fallback	No raw model text crosses a boundary
HTTP contract	Elysia auth, DTOs, idempotency, stable error codes	Client-visible behavior
Security red-line	cross-tenant access, consent race, prompt injection, tool escalation	Hard block and audit
Evaluation	golden product/profile cases and explanation faithfulness	Calibration and regression detection
Crash/recovery	commit before cache, audit failure, duplicate request, later stream reclaim	No duplicate logical effects

13.2 Minimum golden scenarios

- Confirmed boycott + strict ethics policy -> not_recommended and verified alternatives.
- Not boycotted + hard allergy conflict -> not_recommended even if goals fit.
- Not boycotted + strong goal fit -> recommended with evidence and high/moderate confidence.

- Boycott provider unavailable -> unknown/needs_review, never not_boycotted by default.
- Consent absent -> generic product and boycott result only.
- Consent withdrawn between Vault Keeper and commit -> personal decision blocked.
- Storyteller invents a benefit -> Gatekeeper rejects and template fallback is used.
- Two sources disagree -> disagreement is persisted and confidence decreases.
- Cache hit after evidence supersession -> rejected hit and live path or NOT_SURE.

13.3 Metrics without personal leakage

- Request latency, outcome, cache hit/rejected hit, role timeouts, and gate trips.
- Evidence freshness, missing-role rate, disagreement rate, and confidence calibration.
- Provider latency/status, fit dimension coverage, alternative availability, and NOT_SURE rate.
- Tool calls and cost units by role/config version - never prompts containing private feature codes.
- Decision feedback and correction rate using pseudonymous aggregates only.

14. Delivery roadmap



Figure 5. A six-wave path. Each wave is useful on its own and does not depend on unsafe unfinished automation.

Wave	Outcome	Deliverables	Exit evidence
Wave 0	Safety foundation	Approve boycott policy; harden tenant migrations; define consent purposes; create AI context and stable error codes	Historical migration tests; threat model; owner approval
Wave 1	Deterministic MVP	Profile/consent tables, product facts, hard rules, soft fit, decision schema, evidence commit	No LLM; golden scenarios pass; product decision endpoint
Wave 2	Bounded explanation	Storyteller agent with no tools, Zod output, template fallback, faithfulness evaluation	Every sentence traceable to structured input
Wave 3	Alternatives and history	Safe alternative filtering, optional history aggregates, profile cache, confidence calibration	Consent tests; safe alternative invariant
Wave 4	Background knowledge	Versioned claims, validation gate, Redis Streams, outbox/inbox, urgent recalls, DLQ	Crash/retry/reclaim tests; no Store A credentials
Wave 5	Governance	Scorecards, immutable config proposals, human approval, canary, rollback, audit verification	Agents cannot activate; canary stop conditions tested

14.1 Recommended first backlog

Order	Work item	Why first
1	Write the product-decision DTO and outcome policy	Aligns backend, mobile, tests, and product language before implementation
2	Define consent purposes and minimized feature-code registry	Prevents raw-profile architecture from spreading

Order	Work item	Why first
3	Add versioned profile, consent, decision, evidence, and disagreement migrations	Creates durable authority and auditability
4	Create service_boycott_resolution	Unifies existing provider/internal paths and preserves uncertainty
5	Build deterministic Medic and Coach services	Produces a useful MVP without model risk
6	Add authenticated /ai/product-decision with idempotency	Exposes one stable vertical slice
7	Add red-line and integration tests	Proves consent, tenancy, and hard gates
8	Add Storyteller behind a feature flag	Improves UX after truth is stable

15. Definition of done

15.1 A new agent is done when

- Input, output, status, claim, evidence, warning, and fallback schemas are defined and reject unknown fields.
- The role has an explicit purpose, required/optional classification, deadline, budget, and least-privilege tool allowlist.
- Every timeout, dependency loss, invalid output, policy failure, and consent failure has a documented safe outcome.
- The agent cannot access a store or tool outside its declared authority, and denial is audited.
- Unit, contract, security, and failure-path tests pass. Model evaluation exists when an LLM is used.

15.2 The product-fit feature is done when

- Identity, boycott, safety, goal fit, confidence, and final outcome remain separate fields.
- Confirmed boycott behavior follows an approved policy, not prompt improvisation.
- Personal decisions bind to current consent and cannot cross user or tenant boundaries.
- All hard restrictions are deterministic and tested; missing safety data produces NOT_SURE.
- Every material knowledge claim has evidence identity, date, freshness, and version.
- Disagreement is visible and persisted; agent agreement alone cannot create confidence.
- The model can be disabled without disabling truthful deterministic decisions.
- The latest migrations, container build, Elysia contract tests, Redis/Turso integration tests, and rollback procedures pass.
- Logs, metrics, traces, and audit records contain no raw profile, health, prompt secrets, or credentials.

Release rule

FAST when evidence is strong. QUALIFIED when evidence is limited. NOT_SURE when evidence is thin. BLOCKED when safety, privacy, consent, tenancy, schema, or audit integrity fails. AUDITED in every case.

16. Beginner glossary and reading path

Term	Plain meaning in this project
Agent	A bounded role that may be a model, rule set, retrieval step, or workflow component.
Tool	A typed, authorized adapter around an existing backend service.
Orchestrator	The workflow supervisor that plans tasks, applies budgets, and joins results.
Deterministic rule	Code that returns the same result for the same inputs and configuration.
Retrieval	Reading trusted catalog, evidence, profile codes, or history instead of asking a model to remember facts.
Hard gate	A non-negotiable decision that can block, such as consent, allergy, tenant scope, or schema.
Claim	A structured statement from a specialist with a subject, position, evidence, and materiality.
Evidence	A sourced fact carrying identity, dates, freshness, and position.
Confidence	How reliable and complete the evidence is; not the same as user fit.
Coverage	The share of required evaluation dimensions that successfully ran.
Idempotency	Repeating the same request or job does not create a second logical effect.
Outbox/inbox	Durable patterns that connect database writes and Redis work without pretending they share one transaction.
Store A/B/C	Private user data, shared knowledge, and internal/control data respectively.
Canary	A bounded trial of an approved configuration before wider promotion.

16.1 Suggested learning order

- Read the Decision contract and outcome matrix first. It defines what the product promises.
- Trace the current `scan.service.ts`, Boycat provider, and internal boycott-decision service.
- Learn the user tenant, access checks, scan history, and user list to understand Store A isolation.
- Build and test deterministic profile, consent, safety, and fit services before studying agent prompts.
- Learn Zod structured output, VoltAgent tools, and the repository's Agent/Tool patterns.
- Study Redis caching, idempotency, outbox/inbox, and worker recovery only after the synchronous MVP works.
- Study evaluation, privacy-cleared reporting, and configuration governance last.

17. Sources and repository anchors

Primary source: **ZTroop by Sag Ine - Complete Algorithms Book - Final**, 56 pages. The most directly used sections are: non-negotiable invariants; common execution contracts; Consumer AI Army algorithm; cache/retrieval; jobs and workers; hard rules; platform contracts; and acceptance tests.

Repository snapshot anchors:

- `package.json` - Bun, Elysia, Turso/libSQL, Drizzle, Redis, R2, VoltAgent, AI SDK, Google provider, and Zod.
- `src/app.ts` - authenticated controller group and application lifecycle.
- `src/module/main/scan/scan.service.ts` - product resolution, scan history, and current Boycat flow.
- `src/module/main/boycott-provider/*` - encrypted Boycat provider contract.
- `src/module/main/boycott-decision/*` - internal evidence matching and confidence.
- `src/db/main.schema.db.ts` and `src/db/user.schema.db.ts` - existing shared and user-private tables.
- `src/ai/utils.ai.ts` and `src/ai/tool/utils.tool.ts` - current AI foundation and tool context gap.
- `.agents/skills` - required architecture, folder, Agent, and Tool conventions.

Final recommendation

Ship a deterministic product-fit decision service first, then place a tool-less Storyteller model behind it. This delivers immediate user value while keeping boycott evidence, health restrictions, consent, and authorization under testable backend control.